Instalação do VTT Sonho Jovem

WSL 2, Ubuntu e dependências do projeto

Guia de desenvolvimento local · 06/11/2026

Arquivo: `INFO-instalacao-projeto.pdf` · Fonte editável: `INFO-instalacao-projeto.md`.

Este tutorial prepara um computador Windows para executar o VTT Sonho Jovem no Ubuntu pelo WSL 2. Os comandos das aplicações são executados no Linux; o navegador e o Docker Desktop ficam no Windows.

Origem e escopo. O link compartilhado indicado pelo usuário não expôs o conteúdo da conversa durante a consulta. Portanto, este documento é uma consolidação dos arquivos atuais do repositório e das fontes oficiais indicadas ao longo do texto, não uma transcrição da conversa. Link de referência: conversa compartilhada.

Neste guia

Instalação do VTT Sonho Jovem
WSL 2, Ubuntu e dependências do projeto
1. Entenda o ambiente
2. Instale o WSL 2 e o Ubuntu
3. Configure o Docker Desktop
4. Prepare as ferramentas do Ubuntu
5. Clone o projeto e selecione o Node
6. Instale pnpm e as dependências
7. Crie a configuração local
8. Inicie o sistema
9. Verifique o ambiente
10. Prepare os testes de navegador — opcional
11. Abra o projeto no VS Code — opcional
12. Retome e encerre o trabalho
13. Resolva os problemas mais comuns
14. Referências e verificação desta edição

1. Entenda o ambiente

Ferramenta	Onde executar/instalar	Para que serve
WSL 2 + Ubuntu	Windows	Executar as ferramentas Linux
Docker Desktop + Compose	Windows, integrado ao Ubuntu	Iniciar PostgreSQL e MailHog
Git, curl, compiladores e Python	Ubuntu	Clonar e preparar dependências
nvm + Node.js	Usuário do Ubuntu	Usar a versão de Node do projeto
pnpm	Usuário do Ubuntu	Instalar pacotes e iniciar o workspace
VS Code + extensão WSL	Windows	Editar com ferramentas do Ubuntu
Navegador	Windows	Acessar frontend e e-mails locais

Versões exigidas pelo repositório nesta edição:

- Node.js **26.13.0**, definido em `.nvmrc`. `package.json` aceita `>=26.13.0 <27`.
- pnpm **12.18.0**, definido em `package.json`.
- PostgreSQL **17** e MailHog **v1.0.1**, definidos em `compose.yaml`.

Vue, Quasar, PixiJS, NestJS e TypeScript são dependências do workspace: serão instalados pelo pnpm. Não é necessário instalá-los globalmente, nem instalar PostgreSQL diretamente no Ubuntu.

2. Instale o WSL 2 e o Ubuntu

Terminal: PowerShell do Windows, como administrador.

Em Windows compatível, instale o WSL com Ubuntu:

```
wsl --install -d Ubuntu
```

Reinicie o computador quando solicitado. Abra **Ubuntu** pelo menu Iniciar e crie seu usuário e senha Linux. A senha digitada no terminal não mostra caracteres; isso é normal.

Depois, confira e atualize pelo PowerShell:

```
wsl --update
wsl --set-default-version 2
wsl --list --verbose
```

A distribuição usada deve aparecer com **VERSION 2**. Se estiver em 1, use o nome exato listado:

```
wsl --set-version Ubuntu 2
```

Se a instalação ficar parada no download, a alternativa documentada é:

```
wsl --install --web-download -d Ubuntu
```

Se aparecer erro de virtualização, confira se a virtualização de hardware está habilitada na BIOS/UEFI e consulte a solução indicada pela Microsoft para o código de erro recebido.

Fonte: Instalar WSL — Microsoft.

3. Configure o Docker Desktop

Aplicação: Docker Desktop no Windows.

Instale pelo guia oficial para Windows, abra o aplicativo e espere o mecanismo iniciar.

No Docker Desktop:

1. Em **Settings → General**, use o mecanismo baseado em **WSL 2**, quando essa opção aparecer.
2. Em **Settings → Resources → WSL Integration**, habilite a integração com a distribuição Ubuntu escolhida.
3. Aplique as alterações e reinicie o Docker Desktop se solicitado.

Evite misturar esse fluxo com uma instalação separada de Docker Engine dentro do mesmo Ubuntu. O guia usa a integração do Docker Desktop.

Terminal: Ubuntu. Valide cliente, servidor e Compose:

```
docker version
docker compose version
```

`docker version` deve mostrar também os dados do **Server**. Erro ao conectar ao daemon geralmente indica Docker Desktop parado ou integração com a distribuição incorreta.

Fonte: Docker Desktop com WSL 2.

4. Prepare as ferramentas do Ubuntu

Terminal: Ubuntu, com seu usuário Linux normal.

```
sudo apt update
sudo apt install -y git curl ca-certificates build-essential python3
```

O `sudo` é usado aqui para instalar pacotes do sistema. As ferramentas de Node e os comandos do projeto serão executados como usuário normal.

Confira o Git e crie a pasta de trabalho:

```
git --version
mkdir -p ~/projetos
cd ~/projetos
```

Mantenha o repositório no sistema de arquivos Linux, por exemplo `~/projetos/vtt-platform`, para que instalação, permissões e ferramentas trabalhem no mesmo ambiente. Os exemplos seguintes usam esse caminho.

5. Clone o projeto e selecione o Node

Terminal: Ubuntu.

Se ainda não tiver o clone:

```
cd ~/projetos
git clone https://github.com/Co0bY/VTT-Sonho-Jovem.git vtt-platform
cd vtt-platform
```

Se o repositório exigir autenticação, use uma conta com acesso. Se o clone já existir, apenas entre em sua pasta; não substitua os arquivos locais.

Instale o nvm no seu usuário, usando o instalador versionado consultado para este guia:

```
curl -fsSL -o /tmp/vtt-instalar-nvm.sh \
  https://raw.githubusercontent.com/nvm-sh/nvm/v0.41.0/install.sh
bash /tmp/vtt-instalar-nvm.sh
source ~/.bashrc
command -v nvm
```

Na raiz do repositório, instale e selecione a versão indicada em `.nvmrc`:

```
nvm install
nvm use
node --version
```

Resultado esperado nesta edição: `v26.13.0`. Em um novo terminal, execute `nvm use` novamente na raiz do projeto.

Fonte: Instalação do nvm.

6. Instale pnpm e as dependências

Terminal: Ubuntu, na raiz do repositório e com o Node selecionado.

```
npm install --global pnpm@12.18.0
pnpm --version
pnpm install --frozen-lockfile
```

A instalação global acima disponibiliza apenas o gerenciador. Os comandos do workspace usam **pnpm**. O resultado esperado de `pnpm --version` é `12.18.0`.

`--frozen-lockfile` usa as versões registradas no lockfile. Se houver erro de incompatibilidade, confira a versão do pnpm e os arquivos do checkout antes de alterar o lockfile.

Com o Node instalado por nvm, execute essa instalação sem `sudo`. Para problemas de permissão, primeiro confira `command -v node` e `command -v npm`: eles devem apontar para o ambiente do seu usuário em `.nvm`.

Fonte: Instalação do pnpm.

7. Crie a configuração local

Terminal: Ubuntu, na raiz do repositório.

```
test -f .env || cp .env.example .env
```

Esse comando preserva um `.env` já existente. O exemplo do repositório contém a senha de desenvolvimento do PostgreSQL e a configuração SMTP local do MailHog. Os valores principais são:

```
POSTGRES_PASSWORD=vtt_local_dev
WEB_ORIGIN=http://127.0.0.1:9000
SMTP_HOST=127.0.0.1
SMTP_PORT=1025
SMTP_SECURE=false
SMTP_FROM="VTT Sonho Jovem <nao-responda@vtt.local>"
```

Use esses valores somente no desenvolvimento local. O arquivo real `.env` é ignorado pelo Git. O Compose e a API leem a mesma senha; não é necessário configurar uma segunda senha separada na aplicação.

Se já houver um volume do PostgreSQL, alterar `POSTGRES_PASSWORD` no arquivo não muda automaticamente a senha do banco existente. Preserve a configuração correspondente ao volume ou ajuste a credencial do banco de forma planejada.

Os uploads ficam por padrão em `storage/`, na raiz. `STORAGE_ROOT` permite outro caminho. Ao copiar um ambiente já usado, preserve tanto o volume do banco quanto os arquivos de `storage/`.

8. Inicie o sistema

Primeiro, abra o Docker Desktop no Windows. Depois, no Ubuntu:

```
cd ~/projetos/vtt-platform
nvm use
docker compose up -d --wait
pnpm dev
```

O Compose inicia **PostgreSQL e MailHog**. O frontend e a API são iniciados separadamente por `pnpm dev`; mantenha esse terminal em execução enquanto trabalha.

A API inicializa as estruturas do banco no próprio processo de partida. Não existe uma etapa separada de Prisma neste checkout.

Abra no navegador do Windows:

Serviço	Endereço	O que verificar
Frontend	http://127.0.0.1:9000	Tela de entrada e acesso às mesas
API	http://127.0.0.1:3000/health	Saúde da API e conexão com banco
MailHog	http://127.0.0.1:8025	E-mails enviados localmente

No cadastro, informe nome e e-mail. Abra o MailHog, localize a mensagem de primeiro acesso e use o link para definir a senha. O MailHog captura os e-mails de desenvolvimento; não é uma caixa postal real na internet.

9. Verifique o ambiente

Abra outro terminal Ubuntu, na raiz do projeto:

```
docker compose ps
docker compose exec -T postgres pg_isready -U vtt -d vtt
curl -fsS http://127.0.0.1:3000/health
curl -fsS http://127.0.0.1:9000/api/health
```

O PostgreSQL deve estar **healthy**, o MailHog em execução e o `pg_isready` deve indicar que o banco aceita conexões. As duas consultas HTTP devem retornar:

```
{"status":"ok","database":"up"}
```

A consulta pela porta 9000 também verifica o proxy do servidor de desenvolvimento. Para verificar tipos e gerar as compilações:

```
nvm use
pnpm --filter @vtt/web typecheck
pnpm build
```

Esses comandos validam o código; o frontend compilado não substitui automaticamente o processo `pnpm dev` já em uso.

10. Prepare os testes de navegador — opcional

Terminal: Ubuntu, na raiz do projeto e após instalar as dependências.

O teste de navegador usa Chromium via Playwright. Instale o navegador e suas bibliotecas de sistema:

```
pnpm --filter @vtt/web exec playwright install --with-deps chromium
```

A instalação das bibliotecas pode pedir a senha de `sudo`. Use uma distribuição aceita pela versão de Playwright do projeto; confira os requisitos oficiais se houver erro de plataforma.

Com o banco iniciado, execute apenas os testes necessários à mudança:

```
pnpm --filter @vtt/web test
pnpm --filter @vtt/api test:scenes
pnpm --filter @vtt/api test:integration
mkdir -p analise-arquivo
pnpm --filter @vtt/api test:browser
```

Os testes da API usam o banco local e criam dados temporários. O teste de navegador inicia processos próprios e grava capturas em `analise-arquivo/`. Evite executar duas instâncias desse mesmo teste simultaneamente.

Fonte: Navegadores e dependências do Playwright.

11. Abra o projeto no VS Code — opcional

Windows: instale o VS Code e a extensão **WSL**, da Microsoft. **Ubuntu:** abra a pasta do projeto com:

```
cd ~/projetos/vtt-platform
code .
```

Confira que o VS Code indica uma conexão com WSL/Ubuntu. O terminal integrado deve executar Linux e `pwd` deve mostrar o caminho em `/home/.../projetos/vtt-platform`.

Instale as extensões de linguagem no ambiente WSL quando o editor oferecer essa opção. O Git e o pnpm usados no terminal devem ser os instalados no Ubuntu.

Fonte: Desenvolver no WSL com VS Code.

12. Retome e encerre o trabalho

Para retomar, abra Docker Desktop e Ubuntu. No repositório:

```
nvm use
docker compose up -d --wait
pnpm dev
```

Para parar a aplicação, pressione **Ctrl+C** no terminal de `pnpm dev`. Para pausar os contêineres, sem apagar o volume:

```
docker compose stop
```

Quando terminar, você também pode encerrar o Docker Desktop pela interface. Não use `docker compose down -v` para uma pausa: a opção `-v` remove o volume persistente do banco.

Neste computador de referência, a inicialização foi configurada como manual. Em outro computador, as opções de início automático podem ser diferentes; este tutorial não altera serviços do Windows.

13. Resolva os problemas mais comuns

Não abre o frontend em 127.0.0.1:9000

Confira o terminal de `pnpm dev`. Docker em execução, sozinho, não inicia o frontend. Verifique as portas no Ubuntu:

```
ss -ltnp | grep -E ':9000|:3000'
```

Se a porta já estiver ocupada, identifique o processo e encerre somente a instância que você reconhece. Se o frontend responder no Ubuntu, mas não no Windows, confira o encaminhamento de localhost do WSL e o firewall local antes de mudar o endereço de escuta do projeto.

Docker não conecta ou retorna permission denied

Abra o Docker Desktop, confira a integração com a distribuição e reabra o terminal Ubuntu. Rode novamente `docker version`. Este guia pressupõe Docker Desktop com WSL; não use permissões abertas no socket como solução.

nvm ou pnpm não é encontrado

Reabra o Ubuntu ou execute `source ~/.bashrc`, entre no repositório e use `nvm use`. Confira `node --version` e `pnpm --version` antes de reinstalar pacotes.

O banco não fica saudável ou a API falha

```
docker compose ps
docker compose logs --tail=80 postgres
```

Confira o `.env`, a disponibilidade da porta 5432 e a senha associada ao volume existente. A API informa a falha no terminal; não apague o volume para corrigir credenciais de um ambiente com dados úteis.

Chromium não abre nos testes

Erros citando `libnspr4.so`, `libnss3.so` ou `libasound.so.2` indicam bibliotecas de sistema ausentes. Repita a instalação do Playwright com `--with-deps`, descrita no passo 10, e confira a compatibilidade da distribuição.

14. Referências e verificação desta edição

As versões e comandos do projeto foram conferidos em `.nvmrc`, `package.json`, `compose.yaml`, `.env.example`, scripts de `apps/api` e `apps/web`, além dos guias locais de ambiente e retomada. Fontes externas foram consultadas em **06/11/2026**; seus links aparecem junto às etapas correspondentes.

Foram verificados neste ambiente: versões de Node/pnpm, disponibilidade dos contêineres, saúde da API e resposta do frontend. A instalação do Windows/WSL do zero não foi refeita nesta máquina para produzir o documento. Os comandos de instalação são instruções para executar no computador de destino.

Para manter este material, atualize o Markdown e gere novamente o PDF com o mesmo nome. O prefixo **INFO-** foi usado no lugar da numeração sequencial, conforme solicitado.